CraftCV Frontend Autosave Contract

Audience: Frontend developer

Scope: Saving CV edits automatically, restoring them after reload, and showing the last saved time.

This guide documents the current backend contract. The frontend controls when autosave requests are sent; the backend persists each successful write and records a timestamp on the affected CV.

1. Authentication and JSON conventions

All endpoints in this guide require the signed-in user's access token:

```
Authorization: JWT <access-token>
Content-Type: application/json
```

Use the `JWT` scheme. JSON uses camel-case field names, such as `professionalSummary`, `firstName`, and `lastSavedAt`. IDs are UUID strings.

2. What the backend saves

CV-level data and section data use separate resources:

- The CV record stores the title, professional summary, selected template, and IDs of its section records.
- Personal details are stored once on the user's profile and returned as `personalDetail` on a CV.
- Education, experience, skills, certifications, languages, awards, and additional information each have their own endpoints. Their values are saved on those records; the CV stores their IDs.

When an existing linked section is edited or deleted, the backend updates `lastSavedAt` on the CVs that use it. Updating personal details updates the timestamp for the user's CVs. Adding or removing section IDs through a CV update also updates that CV's timestamp.

3. Load a CV when the editor opens

Fetch the CV before filling the editor:

```
GET /api/cvs/{cvId}/
```

Example response shape:

```
{
  "cvId": "cv-uuid",
  "title": "Software Engineer CV",
  "professionalSummary": "Backend developer.",
  "template": "template-uuid",
  "personalDetail": {
    "firstName": "Ada",
    "lastName": "Example"
  },
  "educations": ["education-uuid"],
  "experiences": ["experience-uuid"],
  "skills": ["skill-uuid"],
  "certifications": [],
  "languages": [],
  "awards": [],
  "additionalInformation": [],
```

```
  "createdAt": "2026-09-30T12:00:00Z",
  "updatedAt": "2026-09-30T12:00:00Z",
  "lastSavedAt": "2026-09-30T12:00:00Z"
}
```

The section properties contain IDs, not full section objects. To populate section forms, fetch that user's section collection and select the records whose IDs appear on the CV. Collection endpoints are listed in section 5. `personalDetail` is included on the CV response when the user has created profile details.

If there is no CV yet, `POST /api/cvs/` creates one. The current create path may return `lastSavedAt: null`; treat the CV as persisted after a successful create, and show a timestamp after the first successful CV or related-data update.

4. Save CV-level edits

Use the CV detail `PATCH` endpoint for fields stored directly on the CV and for changing its section associations:

```
PATCH /api/cvs/{cvId}/
```

Example body:

```
{
  "title": "Software Engineer CV",
  "professionalSummary": "Backend developer with cloud experience."
}
```

The request is partial: send only fields that changed. To attach or replace section associations, send the complete ID array for the field being changed:

```
{
  "skills": ["skill-uuid-1", "skill-uuid-2"]
}
```

An explicit empty array removes all associations of that type from the CV but does not delete the section records:

```
{
  "skills": []
}
```

A successful response contains the CV data, including an updated `lastSavedAt`, and a `message` such as `CV saved successfully`. This is the authoritative timestamp for the CV.

5. Save section edits

Create each section through its collection endpoint and retain the ID returned by the backend. For an existing section, send `PATCH` to its detail endpoint. Section records are user-owned and may be referenced by one or more of that user's CVs.

Section	Create endpoint	Update endpoint
Education	`POST /api/cvs/educations/`	`PATCH /api/cvs/educations/{educationId}/`
Experience	`POST /api/cvs/experiences/`	`PATCH /api/cvs/experiences/{experienceId}/`
Skill	`POST /api/cvs/skills/`	`PATCH /api/cvs/skills/{skillId}/`

Section	Create endpoint	Update endpoint
Certification	`POST /api/cvs/certifications/`	`PATCH /api/cvs/certifications/{certificationId}/`
Language	`POST /api/cvs/languages/`	`PATCH /api/cvs/languages/{languageId}/`
Award	`POST /api/cvs/awards/`	`PATCH /api/cvs/awards/{awardId}/`
Additional information	`POST /api/cvs/additional-information/`	`PATCH /api/cvs/additional-information/{additionalInfoId}/`

Example education create body:

```
{
  "institution": "University of Ghana",
  "degree": "BSc",
  "fieldOfStudy": "Computer Science",
  "startDate": "2021-09-01",
  "endDate": null,
  "isCurrent": true,
  "description": "",
  "displayOrder": 0
}
```

After creating a section, include its returned UUID in the matching CV array with `PATCH /api/cvs/{cvId}/` to associate it with the CV. The current API does not combine section creation and CV association into one request.

For removal, use `DELETE` on the section detail endpoint to delete the user-owned record everywhere it is used, or `PATCH` the CV with an updated ID array to detach it from only that CV.

Section create/update/delete responses do not include the CV's `lastSavedAt`. After a successful section write (and any CV association update), fetch `GET /api/cvs/{cvId}/` to refresh the authoritative timestamp. If several saves happen close together, do one refresh after the batch of writes completes.

6. Save personal details

Personal details use a separate profile endpoint:

```
GET   /api/cvs/personal-details/
PATCH /api/cvs/personal-details/
PUT   /api/cvs/personal-details/
```

Prefer `PATCH` for autosave and send only changed fields:

```
{
  "location": "Accra, Ghana"
}
```

The first `PUT` creates the profile; subsequent `PUT`s are also handled as partial updates. Profile changes update the save timestamp on the user's CVs. The profile response itself does not include that CV timestamp, so fetch the CV after the profile write if the UI needs to display the authoritative time.

7. Autosave scheduling

The backend saves only after it receives a successful request. Implement the timer in the frontend. A practical approach is:

1. Mark the editor dirty whenever a field changes.
2. Debounce rapid typing (for example, send after 1 second without input).
3. Add a maximum wait below 30 seconds (for example, 10–15 seconds after the first unsaved change), so continuous typing still triggers a save within the acceptance limit.
4. Keep a save in flight per resource. If the user edits again during that request, queue the newest state and send it after the current request completes. This prevents an older, slower response from overwriting newer edits.
5. Mark the editor saved only after the server returns success. On failure, keep the resource dirty and retry with backoff; show an unsaved/error state.

Do not rely on `beforeunload` or `pagehide` as the main save mechanism. Browsers may cancel requests while closing. Save during editing so a browser close only risks changes made since the last successful request.

For section arrays, preserve stable section IDs and serialize writes to the same record. For association changes, send the latest full ID array so an older queued array cannot restore a removed section.

8. Showing “Last saved”

Use the server-provided `lastSavedAt` from the CV detail response as the source of truth. Convert the ISO timestamp to the user's local time for display, for example “Saved at 2:41 PM”.

Suggested UI states:

State	Display
Dirty or request pending	`Saving…`
Request succeeded and CV response includes timestamp	`Saved at {localized time}`
Related section/profile request succeeded	Refresh the CV, then show its returned timestamp
Request failed	`Couldn't save. Retrying…` or a clear unsaved warning
`lastSavedAt` is null	No server save timestamp is available yet

Do not set the UI's authoritative `lastSavedAt` to the browser clock. It may be used for optimistic presentation, but replace it with the timestamp returned by or fetched from the backend.

9. Error handling

- `400`: validation failed; keep the data dirty and show field errors.
- `401`: try the app's access-token refresh flow once; if it fails, keep local editor state and ask the user to sign in again.
- `404`: the CV or section no longer exists or is not accessible to this user; do not silently mark it saved.
- Network/5xx error: retain dirty state and retry with bounded backoff.

Error bodies may contain field errors, `error`, or `message`; use the status code and parse the available details rather than assuming a single response envelope.

10. Endpoint summary

Purpose	Endpoint
Load CV and `lastSavedAt`	`GET /api/cvs/{cvId}/`
Save CV fields or section ID arrays	`PATCH /api/cvs/{cvId}/`
Load/create personal details	`GET` / `PUT /api/cvs/personal-details/`
Autosave personal details	`PATCH /api/cvs/personal-details/`
Create a section	`POST /api/cvs/{sectionCollection}/`
Update a section	`PATCH /api/cvs/{sectionCollection}/{sectionId}/`
Detach a section from one CV	`PATCH /api/cvs/{cvId}/` with the updated ID array
Delete a section record	`DELETE /api/cvs/{sectionCollection}/{sectionId}/`

All these endpoints require `Authorization: JWT <access-token>`.